中山大學 SUN YAT-SEN UNIVERSITY

国家超级计算广州中心 NATIONAL SUPERCOMPUTER CENTER IN GUANGZHOU

并行程序设计与算法（实验）

6-Pthreads并行构造

吴迪、刘学正

中山大学 计算机学院

实验概要

实验内容

- 动态库的构建与链接使用
- 基于 Pthreads 的 OpenMP parallel for 并行循环实现
- 典型问题的并行化改造：矩阵乘法，热传导平板问题

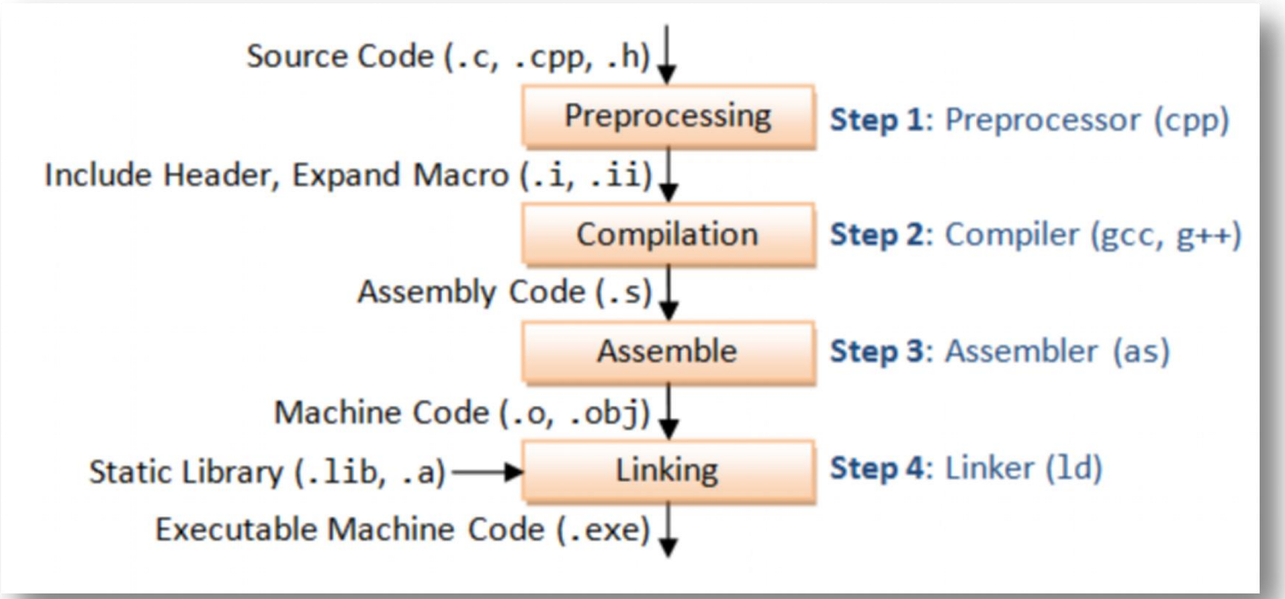
实验目的

- 深入理解C/C++程序编译、链接与构建过程
- 提高Pthreads多线程编程综合能力
- 并行库开发与性能验证

C/C++ 程序的编译

- **从源代码到可执行文件的完整编译流程**
  - **预处理（Preprocessing）**
    - gcc -E hello.c -o hello.i
  - **编译（Compilation）**
    - gcc -S hello.i -o hello.s
  - **汇编（Assemble）**
    - gcc -c hello.s -o hello.o
  - **链接（Linking）**
    - gcc hello.o -o hello

hello.c



hello

gcc hello.c -o hello

参考资料：https://www3.ntu.edu.sg/home/ehchua/programming/cpp/gcc_make.html

从源码到目标代码

预处理、编译和汇编，但不链接

hello.c → `gcc -c hello.c` → hello.o

相互等价，默认已包含stdio.h头文件的路径

`gcc -c hello.c -I/usr/include`

```c
// hello.c
#include <stdio.h>
int x=42;

int main() {
    printf("Hello, world, x = %d\n", x);
    return 0;
}
```

源码 hello.c

已定义的代码段符号（main函数）

未定义的外部符号（需要链接时解析）

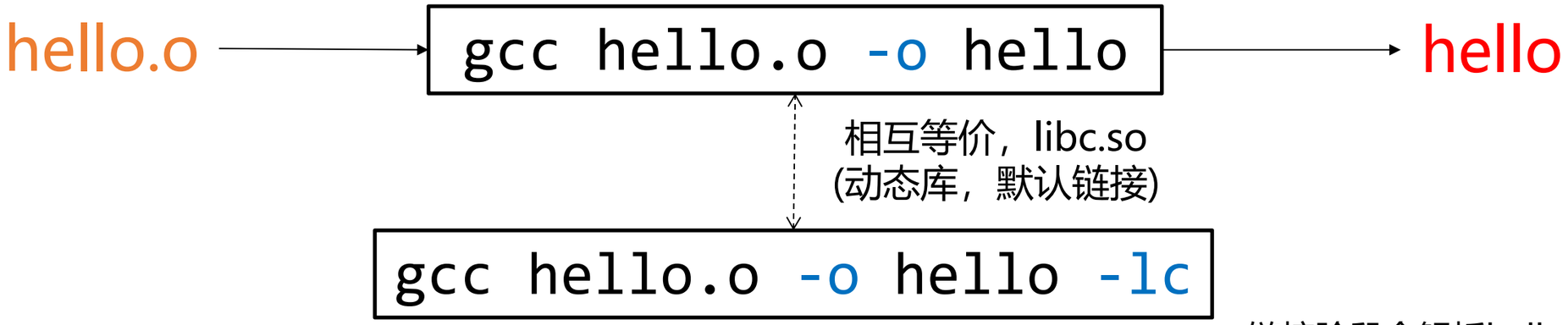
已定义的数据段符号（全局变量x）

```
coder@e7ae7cbac1eb:~/project$ nm hello.o
                 U _GLOBAL_OFFSET_TABLE_
0000000000000000 T main
                 U printf
0000000000000000 D x
```

目标代码符号表

从目标代码到可执行程序

链接过程与标准库的使用



链接阶段会解析hello.o中的外部符号（如printf），默认链接标准 C 库 libc.so，生成可执行文件

```
coder@1db0655e58e9:~/project$ ldd hello
        linux-vdso.so.1 (0x00007ffcf986b000)
        libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f49878c4000)
        /lib64/ld-linux-x86-64.so.2 (0x00007f4987ac4000)
```

查看程序所依赖的动态库

静态和动态链接

静态链接（Static Linking）

- 发生在程序**运行加载前**，链接器将程序用到的库目标代码**完整拷贝**到可执行文件中，完成外部符号解析与绑定
- 生成的可执行文件**体积较大**，运行时不依赖任何外部库文件
- 若库版本更新，必须重新编译链接整个程序

- Linux：lib<library_name>.a
- Windows: lib<library_name>.lib

动态链接（Dynamic Linking）

- 动态链接发生在**程序加载时或运行时**，链接阶段仅做符号引用登记，**不拷贝库代码**，程序运行由系统动态链接器**加载共享库**，完成符号重定位与地址绑定
- 可执行文件**体积更小**，多个进程可共享同一份动态库内存镜像
- 库更新后无需重新编译链接程序

- Linux：lib<library_name>.so
- Windows: lib<library_name>.dll

库的使用：从编译到运行

- 预处理阶段：包含库头文件（-I）

```
gcc … -Iinclude_path …
```

- 链接阶段：链接库文件（-L/-l）

```
gcc … -Llibrary_path -llibrary_name …
```

- 运行阶段：加载动态库（rpath）

```
gcc …-Wl,rpath=library_path …
```

完整命令示例

```
gcc main.c -I./include -L./lib -lmylib -Wl,rpath=./lib -o main
```

动态库制作流程

- **编译位置无关的目标文件**
  - 使用 -fPIC 选项编译源代码

```
gcc -c -fPIC source_file.c …
```

- **链接生成动态库**
  - 使用 -shared 选项
  - 设置 soname

```
gcc -shared -o libfoo.so.x.y
-Wl,-soname,libfoo.so.x …
```

- **安装库文件**
  - 复制到系统库目录（如 /usr/lib64）

```
cp libfoo.so.x.y /usr/lib64
```

- **创建符号链接**
  - 编译时使用libfoo.so
  - 运行时使用libfoo.so.x（soname）

```
ln -s libfoo.so.x.y libfoo.so.x
ln -s libfoo.so.x libfoo.so
```

使用 Makefile 自动构建动态库

项目文件结构

```
.
├── main.c          # 主程序
├── mylib.c         # 库源码
├── mylib.h         # 库头文件
└── Makefile        # 构建脚本
```

编译和运行

变量	含义	示例值
CC	编译器	gcc
CFLAGS	编译选项	-Wall -fPIC
LIB_NAME	库名	mylib
LIB_VERSION	完整版本	1.0
LIB_SONAME_VERSION	soname 主版本	1

编译和运行

```
coder@83905d4117bc:~/project/make_example$ make
gcc -Wall -fPIC -c mylib.c
gcc -shared -Wl,-soname,libmylib.so.1 -o libmylib.so.1.0 mylib.o
ln -sf libmylib.so.1.0 libmylib.so.1
ln -sf libmylib.so.1 libmylib.so
gcc -Wall -fPIC -c main.c
gcc -o main main.o -L. -lmylib -Wl,-rpath,.
coder@83905d4117bc:~/project/make_example$ ./main
Hello from the shared library!
```

示例代码见：make_example.zip

实验内容

- 使用Pthreads构建并行for循环分解、分配、执行机制
  - 模仿OpenMP的omp_parallel_for构造

```
#pragma omp parallel for num_threads(2) schedule(dynamic, chunk)
for (int i = 0; i < N; i++) {
    c[i] = a[i] + b[i];
}
```

  - 生成包含parallel_for函数的动态链接库（.so）文件

```
parallel_for(int start, int end, int inc,
             void *(*functor)(int,void*), void *arg, int num_threads)
```

    - start, end, inc分别为循环的开始、结束及索引自增量
    - functor为函数指针，定义了每次循环所执行的内容
    - arg为functor的参数指针，给出了functor执行所需的数据
    - num_threads为期望产生的线程数量

实验内容

- 使用Pthreads构建并行for循环分解、分配、执行机制
  - 对于以下functor及参数定义

```
struct functor_args {
    float *A, *B, *C;
};

void *functor(int idx, void* args){
    functor_args *args_data = (functor_args*) args;
    args_data->C[idx] = args_data->A[idx] + args_data->B[idx];
}
```

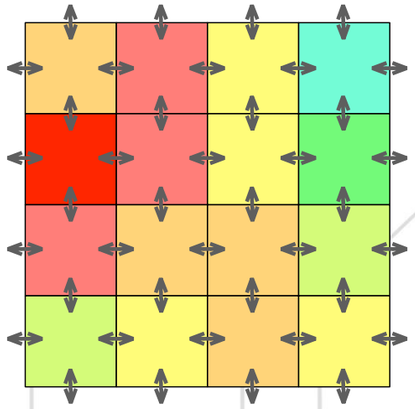
  - 如下代码应产生两个线程，对两个包含10个元素的数组并行求和

```
functor_args args = {A, B, C};
parallel_for(0, 10, 1, functor, (void*)&args, 2)
```

  - 要求：完成parallel_for函数实现，并以矩阵乘法为例进行验证

实验内容

- Pthreads并行应用
  - 使用自定义的parallel_for替代heated_plate_openmp中的并行构造
  - 该应用模拟规则网格上的热传导，每次循环中对邻域内热量平均
    - $w_{i,j}^{t+1} = \frac{1}{4}(w_{i-1,j-1}^{t} + w_{i-1,j+1}^{t} + w_{i+1,j-1}^{t} + w_{i+1,j+1}^{t})$
  - 测试不同线程、调度方式下的程序并行性能
  - 与原始heated_plate_openmp.c实现进行对比



Questions?

